Chapter_4_Presentation_Layer_ST1

Chapter 4: The Presentation Layer (Layer 6) — Encoding, MIME & Markup

> **Scope note:** For **Semester Test 1**, Layer 6 is examined **excluding ASN.1** and excluding "Remaining layer 6 issues" (compression, encryption, data exchange utilities). This file covers everything examinable for ST1: character sets, UTF-8 mechanics, markup languages, non-textual data, and MIME. **ASN.1 is covered in the separate `Layer_6.md` file** (ST2 scope). The "remaining layer 6 issues" section is out of scope for both tests.

---

1. Introduction to the Presentation Layer

The **presentation layer (Layer 6)** sits between the **Session (Layer 5)** and **Application (Layer 7)** layers. Its purpose is to deal with **data representation** — encoding and decoding information so that it is correctly represented in transit and accurately reconstructed at the destination.

Think of it as a **codec** (coder/decoder): it encodes data at the sender and decodes it at the receiver.

> **MCQ fact:** An example of a header field added at Layer 6: `UTF-8` (or any other character encoding designation).

> **MCQ fact (Encryption trap):** On which ISO OSI layer should encryption be placed? **It depends on factors not mentioned in the question; all layers from 3 to 7 are options.** *(Distractor: "Layer 6 only" — wrong.)*

> **MCQ fact (MIME layer):** If MIME is deemed to be a data communications protocol, it fits best on **Layer 6** (the presentation layer).

In modern TCP/IP, Layer 6 functionality is often absorbed into the application layer protocol itself (SMTP, HTTP, etc. define their own encoding rules).

---

2. Representing Text and Character Sets

2.1 Standard vs. Extended ASCII

Standard ASCII uses **7 bits** (128 characters). All standard digits, Latin letters, and common punctuation are covered.

Extended ASCII (e.g., ISO-8859-1) uses **8 bits** to add 128 extra characters. Critically, different vendors/standards use the upper 128 positions differently.

> **MCQ fact (2023 ST2 Q1g / Extended ASCII trap):** Suppose computers A and B both use "extended ASCII", but there is no character conversion mechanism between them. What challenges will they experience?
>
> - They will **NOT** misinterpret standard digits, the 26 Latin letters, or standard punctuation (these are identical across all ASCII variants — they are in the lower 128).
> - They will **NOT** misinterpret markup text like `&euml;` (markup uses only standard ASCII characters: `&`, letters, `;`).
> - The real problem: the **upper 128 characters** (e.g., `ë`, `é`) will be **misinterpreted** because different extended ASCII tables map different characters to those positions.
> - Answer: **None of the listed obvious problems will occur** (standard text and markup arrive fine), **but other (severe) challenges exist** (extended characters are misinterpreted).

Other character codes:

- **EBCDIC** (Extended Binary Coded Decimal Interchange Code) — used on IBM mainframes. Incompatible with ASCII.
- **Fieldata** — older military/government encoding.
- **Unicode** — the universal character set. Every character has a unique **code point** expressed as `U+HHHH`.

2.2 UTF-8 Mechanics

UTF-8 is the dominant variable-length encoding for Unicode.

Byte structure rules (MUST memorise):

Byte count	Leading bits	Pattern	Available bits
1 byte	`0`	`0xxxxxxx`	7 bits
2 bytes	`110`	`110xxxxx 10xxxxxx`	11 bits
3 bytes	`1110`	`1110xxxx 10xxxxxx 10xxxxxx`	16 bits
4 bytes	`11110`	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`	21 bits
Trailing octet	`10`	`10xxxxxx`	continuation

> **MCQ fact (Length from first byte):** If the first octet of a UTF-8 character is `193` in decimal: `193` in binary = `11000001`. Starts with `110` → character is **2 octets** long.

> **MCQ fact (Capacity, 4-byte):** How many Unicode characters can be represented in UTF-8 using exactly four bytes? Pattern: `11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`. Available bits: 3 + 6 + 6 + 6 = **21 bits** = **2^{21}** characters. *(Answer: n = 21.)*

> **MCQ fact (Self-synchronisation):** UTF-8 is **self-synchronising**. A trailing octet (`10xxxxxx`) that is not preceded by a valid leading multi-byte octet is simply skipped. The stream re-syncs at the next valid leading byte.

Example 1 — Counting characters in a hex sequence:
Sequence: `46 33 EA A7 98 20`

- `46` = `01000110` → starts with `0` → **1-byte char**
- `33` = `00110011` → starts with `0` → **1-byte char**
- `EA` = `11101010` → starts with `1110` → **3-byte char** (next two bytes `A7`, `98` are `10xxxxxx` → valid trailers)
- `20` = `00100000` → starts with `0` → **1-byte char**
  Total: **4 characters**.

Example 2 — Encoding to hex:
The character `Æ` is Unicode `U+00C6`. In binary: `11000110`.
UTF-8 2-byte pattern: `110xxxxx 10xxxxxx`. Insert `11000110` bits:

- Available bits: `11 000110` → `110 00011` `10 000110` → `0xC3 0x86`.
  So `Æ` in UTF-8 = `C3 86`.

Example 3 — Encoding `U+0410` (Cyrillic A):
`U+0410` binary = `100 000001 0000` = needs 2 bytes.
`110 10000` `10 010000` → `D0 90`.

> **MCQ fact (D7 E5 test):** Consider the byte sequence `D7 E5`. Is this valid UTF-8?
>
> - `D7` = `11010111` → starts with `110` → 2-byte leading byte. Requires one trailing `10xxxxxx` byte.
> - `E5` = `11100101` → starts with `1110` → this is a **3-byte leading byte**, NOT a valid trailing byte.
>
> Therefore, `D7 E5` is **not a valid UTF-8 encoding**. *(2025 ST2 Q1h answer: E — not valid.)*

2.3 Content Negotiation

Applications negotiate character sets using `q` (quality/preference) parameters in HTTP headers.

Example — `Accept-Charset: iso-8859-1, utf-8, utf-16,* ;q=0.1 ` The wildcard with quality 0.1 means any other charset is acceptable (though low priority). If the server supports` *Shift_JIS* `, it may use it – the ` ` covers it.

> **MCQ fact (2025 ST2 Q3 — Wildcard trap):** With `Accept-Charset: iso-8859-1, utf-8, utf-16, *;q=0.1`, if the server supports `iso-8859-1`, `utf-8`, and `Shift_JIS`, the response **may be encoded using any of the encodings listed** (including `Shift_JIS` via the wildcard). *(Distractor: "only iso-8859-1 or utf-8" — wrong;* `` includes Shift_JIS.)*

---

3. Markup Languages and Grammars

Markup languages embed structural or presentational information in text.

- **Old style (presentational):** `<u>underline</u>` — specifies *how* to display.
- **Modern style (semantic/intent):** `<em>emphasize</em>` — specifies *what the content is*, letting the renderer decide how to present it.

> **MCQ fact:** The HTTP protocol is specified using **ABNF** (Augmented Backus-Naur Form). *(Distractor: EBNF — wrong. HTTP uses ABNF, not EBNF.)*

Application-oriented content:

- **SOAP** (Simple Object Access Protocol) — enables structured information exchange between distributed objects; primarily used for Web Services.
- **XBRL** — eXtensible Business Reporting Language; used for financial reporting.

---

4. Non-Textual Data

Binary data has no inherent meaning — its interpretation depends on how the presentation layer decodes it (e.g., the same bytes could be a video frame or an image, depending on the codec).

EXIF data: Metadata embedded in photographs (shutter speed, geo-tags, camera model). Acts like markup for binary image data, giving semantic meaning.

---

5. MIME (Multipurpose Internet Mail Extensions)

MIME extends email and HTTP to carry arbitrary content types (images, video, attachments, etc.). If MIME is treated as a protocol, it fits on **Layer 6**.

5.1 MIME Types

A MIME type has the form `type/subtype`. Valid top-level types:

Valid Type	Examples
`text`	`text/plain`, `text/html`
`image`	`image/jpeg`, `image/png`
`audio`	`audio/mpeg`
`video`	`video/mp4`
`application`	`application/pdf`, `application/json`
`multipart`	`multipart/mixed`, `multipart/alternative`
`message`	`message/rfc822`
`model`	`model/vrml`
`example`	For documentation only

> **MCQ fact (2025 ST2 Q4):** Which of the following is **not** a valid MIME type? `binary` is not a valid MIME top-level type. (`example`, `image`, `message`, and `model` are all valid.)

`multipart/mixed`**:** The email or message consists of several parts of **different** types (e.g., an HTML body plus a JPEG attachment).
`multipart/alternative`**:** Multiple versions of the same content (e.g., plain text and HTML version of the same email body).

> **MCQ fact (2025 ST2 Q6):** An email consisting of an HTML message and a JPG image (both included) uses MIME type `multipart/mixed` to describe the entire message. *(Distractor: `text/html` — describes only one part, not the whole.)*

> **MCQ fact (2025 ST2 Q5):** A message consisting of JPG images embedded in HTML text uses MIME type `multipart/mixed`. *(HTML text + images = multiple different types → mixed.)*

5.2 Boundary Markers

In a `multipart/*` message, parts are separated by a **boundary string** specified in the Content-Type header:

```
Content-Type: multipart/mixed; boundary="----=_NextPart_000"
```

Each part is preceded by `--` + the boundary string. The boundary string must be chosen so it cannot appear accidentally in any part.

5.3 `Content-Transfer-Encoding`

Specifies how the content has been further encoded before transmission (on top of its native encoding).

Value	Meaning
`7bit`	No transformation; content uses only 7-bit ASCII
`8bit`	No transformation; content may use all 8 bits
`binary`	No transformation; no line-length constraints
`base64`	Binary data encoded as 7-bit ASCII (safe for all transport)
`quoted-printable`	Human-readable encoding; non-printable bytes as `=HH`

> **MCQ fact (2023 ST1 / 2025 tests):** To transfer a PNG image via SMTP (which was designed for 7-bit ASCII text), the most appropriate `Content-Transfer-Encoding` is `base64`. *(SMTP cannot handle raw binary; base64 maps binary to safe 7-bit ASCII.)*

Example 1 — base64:
The string `abc` (8-bit ASCII: `61 62 63`) encodes to `YWJj` in base64. Length increases (3 bytes → 4 chars) because 8-bit values are mapped to 6-bit indices.

Example 2 — quoted-printable:

- `abc` stays as `abc` (printable ASCII).
- Bytes `01 02 03` (non-printable) become `=01=02=03`.
- The literal string `a=b` becomes `a=3Db` (the `=` sign itself must be escaped as `=3D`).

5.4 MIME in Email — `Content-Type` Header

```
MIME-Version: 1.0
Content-Type: multipart/mixed; boundary="----=_NextPart"

------=_NextPart
Content-Type: text/html
Content-Transfer-Encoding: quoted-printable
```

```
<html>...

------=_NextPart
Content-Type: image/jpeg
Content-Transfer-Encoding: base64

/9j/4AAQSkZJRgAB...
------=_NextPart--
```

> **MCQ fact:** The first **blank line** in an email message separates the **header** from the **body** (payload). This is also true in HTTP.

---

6. Summary: Layer 6 Quick-Reference

Topic	Key fact
Layer position	Between Layer 5 (session) and Layer 7 (application)
Header field example	`UTF-8`, `base64`, `multipart/mixed`
Encryption layer	Depends — all of L3–L7 are valid answers
MIME as protocol	Layer 6
Standard ASCII bits	7
Extended ASCII bits	8
UTF-8 self-sync	Yes — invalid trailing octets are skipped
4-byte UTF-8 capacity	2^{21} characters (n = 21)
PNG over SMTP encoding	`base64`
Not a valid MIME type	`binary`
Multipart email type	`multipart/mixed`
HTTP grammar format	ABNF (not EBNF)

Examinable Practice Questions

Multiple Choice

Q1. If MIME is deemed to be a data communications protocol, it fits best on layer ... of the ISO OSI model:

A. 7 B. 6 C. 5 D. 4 E. 3

Q2. Which character set is assumed to be understood by all parties involved in an HTTP exchange?
A. ASCII B. EBCDIC C. Unicode D. UTF-8 E. ISO-8859-1

Q3. Consider the HTTP header `Accept-Charset: iso-8859-1, utf-8, utf-16, *;q=0.1`. Suppose the server supports `iso-8859-1`, `utf-8`, and `Shift_JIS`. The response may then be encoded using:
A. iso-8859-1 B. utf-8 C. Shift_JIS D. Any of the encodings listed E. One or two of the encodings listed

Q4. Which of the following is NOT a valid MIME type?
A. binary B. example C. image D. message E. model

Q5. Suppose a message consists of JPG images embedded in HTML text. Which MIME type and subtype will be used to describe the message?
A. text/plain B. text/html C. image/jpeg D. multipart/mixed E. multipart/alternative

Q6. Suppose an email consists of an HTML message and a JPG image (both components). Which MIME type describes the entire email message?
A. application/email B. application/rfc822 C. multipart/alternative D. multipart/mixed E. text/html

Q7. How many Unicode characters can, in principle, be represented in UTF-8 using exactly four bytes? If expressed as 2^n, what is n?
A. 14 B. 20 C. 21 D. 22 E. 27

Q8. Consider the byte sequence `D7 E5`. The Unicode character represented by this UTF-8 encoding is:
A. An accented character B. A Greek character C. An emoji D. A punctuation mark E. `D7 E5` is not a valid UTF-8 encoding

Q9. Which `Content-Transfer-Encoding` would be most appropriate to transfer a PNG image via SMTP?
A. 8bit B. binary C. quoted-printable D. base64 E. 7bit

Q10. On which ISO OSI layer should encryption be placed?
A. 7 B. 6 C. 4 D. 3 E. It depends on factors not mentioned in the question; all layers from 3 to 7 are options

Q11. Suppose computers A and B both use "extended ASCII" with no character conversion between them. Which of the following challenges will they experience?
A. Standard digits will be misinterpreted. B. Markup text like `&euml;` will be received incorrectly. C. Standard punctuation may not be communicated correctly. D. No challenges, since they use

the same character encoding. E. None of the problems above will occur (but other severe challenges exist).

Q12. The HTTP protocol specification uses which grammar formalism?
A. BNF B. EBNF C. ABNF D. XML Schema E. ASN.1

Long-Form & Calculation

Q13. UTF-8 encoding. [6]

- a) Encode the following Unicode code points in UTF-8 hex. Show your working.
  - (i) `U+00C6` (Æ)
  - (ii) `U+1040B` (a 4-byte character)
- b) How many Unicode characters can be represented in UTF-8 using exactly three bytes? Express as 2^n and state n.

Q14. UTF-8 decoding. [4]
Convert the following UTF-8 hex byte sequences to Unicode code points:

- a) `C3 86`
- b) `F0 90 AB 90`
- c) `E2 B2 A0`

Q15. Extended ASCII trap. [3]
Computers A and B both use extended ASCII (e.g., ISO-8859-1) but there is no character conversion mechanism. Assume every byte transmitted arrives correctly. Describe the challenges (if any) that will arise when: (i) A sends the character `Z` to B. (ii) A sends the character `ë` to B. (iii) A sends the markup text `&euml;` to B.

Memo / Answer Key

MCQ Answers

A1. B (Layer 6 — presentation layer).
A2. A (ASCII — the one character set that HTTP assumes all parties understand).
A3. D (Any of the encodings — the `*;q=0.1` wildcard covers Shift_JIS).
A4. A (`binary` is not a valid MIME top-level type).
A5. D (`multipart/mixed` — HTML + images = multiple different types).
A6. D (`multipart/mixed` — HTML message body + JPG attachment).
A7. C (n = 21; 4-byte pattern has 3+6+6+6 = 21 available bits).
A8. E (`D7 E5` is not valid — `D7` starts with `110` indicating a 2-byte char, but `E5` starts with

`1110`, which is a 3-byte leading byte, not a valid trailing byte).
A9. D (`base64` — converts binary to 7-bit ASCII safe for SMTP).
A10. E (It depends — all layers 3 to 7 are options).
A11. E (None of the listed problems will occur — standard text arrives fine — but other severe challenges do exist: extended characters above position 127 will be misinterpreted).
A12. C (ABNF — Augmented Backus-Naur Form).

Long-Form Answers

A13. UTF-8 encoding:

(i) U+00C6 (Æ):
`U+00C6` = `0x00C6` = binary `0000 0000 1100 0110` = `1100 0110` (8 bits).
Fits in 2-byte UTF-8 range (U+0080 to U+07FF). Pattern: `110xxxxx 10xxxxxx`.
Insert 11 bits `000 1100 0110`: `110 00011 10 000110` → `C3 86`.

(ii) U+1040B:
`U+1040B` requires 4 bytes (> U+FFFF). Binary: `0001 0000 0100 0000 1011` = 21 bits.
Pattern: `11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`.
21 bits: `000 010000 010000 001011`.
→ `11110 000` `10 010000` `10 010000` `10 001011`
→ `F0 90 90 8B`.

(b) 3-byte capacity:
Pattern: `1110xxxx 10xxxxxx 10xxxxxx`. Available bits: 4+6+6 = **16 bits** → n = **16** → 2^{16} = 65,536 characters.

A14. UTF-8 decoding:

(a) C3 86:
`C3` = `11000011` → 2-byte leading (`110`), data bits: `00011`.
`86` = `10000110` → trailing, data bits: `000110`.
Combined: `00011 000110` = `0000 1100 0110` = `0x00C6` → **U+00C6** (Æ).

(b) F0 90 AB 90:
`F0` = `11110000` → 4-byte leading, data bits: `000`.
`90` = `10010000` → trailing, data bits: `010000`.
`AB` = `10101011` → trailing, data bits: `101011`.
`90` = `10010000` → trailing, data bits: `010000`.
Combined: `000 010000 101011 010000` = `0001 0000 1010 1101 0000` = `0x10AB0` → **U+10AB0**.

(c) E2 B2 A0:
`E2` = `11100010` → 3-byte leading, data bits: `0010`.

`B2` = `10110010` → trailing, data bits: `110010`.
`A0` = `10100000` → trailing, data bits: `100000`.
Combined: `0010 110010 100000` = `0010 1100 1010 0000` = `0x2CA0` → **U+2CA0**.

A15. Extended ASCII challenges:

- (i) `Z` — no challenge. `Z` (ASCII 90) is in the lower 128 characters, identical in all ASCII variants.
- (ii) `ë` — **severe challenge**. `ë` is in the upper 128 (e.g., position 235 in ISO-8859-1). Computer B may map that byte to a completely different character if it uses a different extended ASCII variant.
- (iii) `&euml;` — no challenge. The markup uses only standard ASCII characters (`&`, `e`, `u`, `m`, `l`, `;`), all in the lower 128.